Student Placement Progress Monitoring and Analytics System

ALFRED PROJECT BRAIN — SOURCE OF TRUTH

This PDF is the human-readable project specification. The repository file BRAIN.md is the authoritative machine-readable source of truth.

BRAIN.md

Student Placement Progress Monitoring and Analytics System

Alfred Project Brain — Source of Truth

0. AI BUILD CONTRACT

This file is the primary source of truth for the project.

When an AI coding agent works on this repository:

1. Read `BRAIN.md` completely before making changes.
2. Read `docs/Project-Brain.pdf` when architectural context or requirements need confirmation.
3. Follow the master build prompt in `prompts/master-build-prompt.md`.
4. Do not invent requirements that conflict with this file.
5. Do not rewrite working modules unnecessarily.
6. Work in small, testable phases.
7. After every phase, verify compilation/build and update documentation.
8. Never expose passwords, JWT secrets, API keys, or database credentials in source control.
9. Preserve existing API contracts unless a change is explicitly required.
10. If a requirement is ambiguous, choose the simplest maintainable implementation and document the decision.

---

1. PROJECT IDENTITY

Title

Student Placement Progress Monitoring and Analytics System

One-line description

A centralized web platform that monitors the complete placement lifecycle of students from eligibility and application through interview rounds and final selection, with scheduling, resources, notifications, analytics, and CSV reporting.

Review-2 description

"A centralized platform that monitors the complete placement lifecycle of students, from application submission to final selection, while providing interview scheduling, preparation resources, progress tracking, analytics, and report generation."

---

2. PROBLEM STATEMENT

Placement cells often manage student recruitment information through spreadsheets, manual records, email, and messaging applications. This makes it difficult to track applications, interview schedules, round-wise results, student activity, and final placement statistics.

The system solves this by providing one centralized application for students and placement officers.

---

3. OBJECTIVES

• Monitor student placement activities.

• Track recruitment progress round by round.

• Manage company and eligibility information.

• Manage interview schedules.

• Provide preparation resources.

• Notify students about important placement events.

• Generate placement analytics.

• Export placement reports as CSV.

• Maintain auditable application status history.

---

4. USERS AND ROLES

Student

Can:

• Register and log in.

• Maintain profile.

• View eligible companies.

• Apply to eligible companies.

• View application status.

• View round-wise progress.

• View interview schedules.

• Download preparation resources.

• View notifications.

Cannot:

• Create companies.

• Change eligibility rules.

• Change application status manually.

• Manage other students.

Placement Officer / Admin

Can:

• Log in securely.

• Manage students.

• Add/edit/delete companies.

• Define eligibility criteria.

• View applications.

• Shortlist students.

• Schedule interview rounds.

• Update round results.

• Upload preparation resources.

• Send announcements.

• View analytics.

• Export CSV reports.

---

5. CORE WORKFLOW

Student lifecycle:

Eligible

-> Applied

-> Shortlisted

-> Aptitude Cleared

-> Technical Cleared

-> HR Cleared

-> Selected

Alternative terminal state:

Any applicable stage -> Rejected

Rules:

• A student must satisfy company eligibility before applying.

• A student cannot apply twice to the same company for the same drive.

• Status changes must be recorded in status history.

• Admin controls official round results.

• Selected and rejected are terminal application outcomes unless an admin explicitly reopens an application.

• Dates and times must be stored consistently.

---

6. FUNCTIONAL MODULES

Module A — Authentication

Student:

• Register

• Login

• Logout

• JWT-based session

• Password hashing

Admin:

• Secure admin login

• Role-based authorization

Module B — Student Profile

Fields:

• studentId

• registrationNumber

• name

• email

• passwordHash

• department

• cgpa

• phone

• createdAt

• updatedAt

Module C — Company Management

Fields:

• companyId

• companyName

• description

• eligibilityCgpa

• eligibleDepartments

• packageAmount

• applicationDeadline

• driveDate

• active

• createdAt

• updatedAt

Module D — Applications

Fields:

• applicationId

• studentId

• companyId

• appliedDate

• currentStatus

Required operations:

• Apply

• View own applications

• Admin view/filter applications

• Admin update progress

• Prevent duplicate application

Module E — Interview Scheduling

Fields:

• interviewId

• companyId

• roundName

• interviewDate

• venue

• meetingLink (optional)

• instructions

• createdAt

A schedule should be visible to relevant students.

Module F — Preparation Resources

Fields:

• resourceId

• companyId (nullable for general resources)

• title

• description

• fileUrl

• resourceType

• uploadedAt

Support PDF resources initially.

Module G — Notifications

Fields:

• notificationId

• studentId (nullable for global announcement)

• title

• message

• type

• isRead

• createdAt

Types:

• INTERVIEW_REMINDER

• RESULT_UPDATE

• COMPANY_ANNOUNCEMENT

• RESOURCE_UPDATE

• GENERAL

Email integration is optional and should be implemented behind a service interface.

Module H — Status History

Fields:

• historyId

• applicationId

• previousStatus

• currentStatus

• updatedBy

• updatedAt

• remarks

Module I — Analytics and Reports

Dashboard metrics:

• Total students

• Total companies

• Total applications

• Shortlisted students

• Selected students

• Rejected students

• Placement percentage

Charts:

• Company-wise applications/selections

• Department-wise selections

• Status distribution

• Company package overview

CSV:

• Applied students

• Selected students

• Company-wise statistics

• Department-wise statistics

---

7. DATABASE DESIGN

students

• student_id PK

• reg_no UNIQUE

• name

• email UNIQUE

• password_hash

• department

• cgpa

• phone

• created_at

• updated_at

companies

• company_id PK

• company_name

• description

• eligibility_cgpa

• eligible_departments

• package_amount

• application_deadline

• drive_date

• active

• created_at

• updated_at

applications

• application_id PK

• student_id FK -> students.student_id

• company_id FK -> companies.company_id

• applied_date

• status

• UNIQUE(student_id, company_id)

interviews

• interview_id PK

• company_id FK -> companies.company_id

• round_name

• interview_date

• venue

• meeting_link

• instructions

• created_at

resources

• resource_id PK

• company_id FK -> companies.company_id, nullable

• title

• description

• file_url

• resource_type

• uploaded_at

notifications

• notification_id PK

• student_id FK -> students.student_id, nullable

• title

• message

• type

• is_read

• created_at

status_history

• history_id PK

• application_id FK -> applications.application_id

• previous_status

• current_status

• updated_by

• remarks

• updated_at

Use proper foreign keys, indexes, constraints, and timestamps.

---

8. STATUS ENUM

Use a controlled enum/value set:

ELIGIBLE

APPLIED

SHORTLISTED

APTITUDE_CLEARED

TECHNICAL_CLEARED

HR_CLEARED

SELECTED

REJECTED

Do not store arbitrary status strings from the frontend.

---

9. TECHNOLOGY ARCHITECTURE

Frontend

• React.js

• React Router

• Axios

• Bootstrap 5

• Chart library such as Recharts

• Functional components and hooks

Backend

• Java

• Spring Boot 3

• Spring Web

• Spring Data JPA

• Spring Security

• JWT

• Bean Validation

• Maven

Database

• MySQL

Reports

• Server-side CSV generation

Notifications

• Internal notification system required

• Email API optional

---

10. REPOSITORY STRUCTURE

Student-Placement-System/

|

|-- BRAIN.md

|-- README.md

|

|-- frontend/

| |-- package.json

| |-- src/

| |-- components/

| |-- layouts/

| |-- pages/

| | |-- auth/

| | |-- student/

| | |-- admin/

| |-- services/

| |-- hooks/

| |-- utils/

| |-- assets/

| |-- App.jsx

| |-- main.jsx

|

|-- backend/

| |-- pom.xml

| |-- src/main/java/com/placement/

| |-- controller/

| |-- service/

| |-- repository/

| |-- entity/

| |-- dto/

| |-- security/

| |-- config/

| |-- exception/

| |-- util/

| |-- src/main/resources/

| |-- application.properties

|

|-- database/

| |-- schema.sql

| |-- sample_data.sql

|

|-- docs/

| |-- Project-Brain.pdf

| |-- API-Documentation.md

| |-- Database-Design.md

|

|-- prompts/

| |-- master-build-prompt.md

|

|-- uploads/

| |-- resources/

|

Do not commit real uploaded files or secrets.

---

11. FRONTEND PAGES

Public

• Login

• Student Registration

Student

• Student Dashboard

• Profile

• Eligible Companies

• Company Details

• My Applications

• Application Progress

• Interview Schedule

• Preparation Resources

• Notifications

Admin

• Admin Dashboard

• Student Management

• Company Management

• Application Management

• Interview Management

• Resource Management

• Notifications/Announcements

• Analytics

• Reports

UI requirements:

• Responsive

• Clean academic/professional design

• Consistent navigation

• Loading states

• Empty states

• Error states

• Form validation

• Confirmation dialogs for destructive actions

• Status badges

• Search/filter/sort where useful

---

12. BACKEND API GROUPS

Authentication:

• POST /api/auth/register

• POST /api/auth/login

Students:

• GET /api/students/me

• PUT /api/students/me

• GET /api/admin/students

Companies:

• GET /api/companies

• GET /api/companies/{id}

• POST /api/admin/companies

• PUT /api/admin/companies/{id}

• DELETE /api/admin/companies/{id}

Applications:

• POST /api/applications

• GET /api/applications/me

• GET /api/admin/applications

• PUT /api/admin/applications/{id}/status

• GET /api/applications/{id}/history

Interviews:

• GET /api/interviews/me

• GET /api/admin/interviews

• POST /api/admin/interviews

• PUT /api/admin/interviews/{id}

• DELETE /api/admin/interviews/{id}

Resources:

• GET /api/resources

• POST /api/admin/resources

• DELETE /api/admin/resources/{id}

Notifications:

• GET /api/notifications

• PUT /api/notifications/{id}/read

• POST /api/admin/notifications/announcement

Analytics:

• GET /api/admin/analytics/summary

• GET /api/admin/analytics/company-wise

• GET /api/admin/analytics/department-wise

• GET /api/admin/reports/applications.csv

• GET /api/admin/reports/selected.csv

Use DTOs rather than exposing JPA entities directly.

---

13. SECURITY REQUIREMENTS

• Passwords must be hashed.

• JWT secret must come from environment configuration.

• Never hard-code secrets.

• Student endpoints must only expose the logged-in student's private data.

• Admin endpoints require ADMIN role.

• Validate all request data.

• Enforce authorization at backend level; frontend hiding is not security.

• Configure CORS for the frontend origin.

• Return safe error messages.

---

14. ELIGIBILITY LOGIC

A student is eligible when:

• Student CGPA >= company minimum CGPA.

• Student department is included in company eligible departments.

• Application deadline has not passed.

• Company is active.

Eligibility must be calculated by backend.

Frontend can display eligibility but must never be the final authority.

---

15. ANALYTICS DEFINITIONS

Placement percentage:

selected students / total eligible students * 100

Application count:

number of unique applications.

Selection count:

number of applications whose current status is SELECTED.

Department statistics:

aggregate applications and selections by department.

Company statistics:

aggregate applications, shortlisted, selected, rejected by company.

Handle division by zero safely.

---

16. FILE UPLOAD RULES

For PDF resources:

• Validate file type.

• Validate file size.

• Generate safe server-side filenames.

• Never execute uploaded files.

• Store only metadata in database.

• Keep physical files outside source-controlled code where practical.

---

17. ERROR HANDLING

Backend:

• Global exception handler

• Validation error responses

• Resource-not-found response

• Unauthorized/forbidden responses

• Duplicate application response

• Invalid status transition response

Frontend:

• User-friendly error messages

• Retry where appropriate

• No blank screens on API failure

---

18. DEVELOPMENT PHASES

Phase 1 — Foundation

• Repository structure

• Spring Boot setup

• React setup

• MySQL connection

• Environment configuration

Phase 2 — Database

• Tables

• Relationships

• Constraints

• Seed data

Phase 3 — Authentication

• Registration

• Login

• JWT

• Roles

• Protected routes

Phase 4 — Student

• Profile

• Eligible companies

• Apply

• My applications

Phase 5 — Admin

• Companies

• Eligibility

• Applications

• Student management

Phase 6 — Progress

• Status updates

• Status history

• Round-wise progress UI

Phase 7 — Interviews and Resources

• Scheduling

• Resource upload/download

• Student schedule/resources

Phase 8 — Notifications

• In-app notifications

• Optional email

Phase 9 — Analytics

• Dashboard

• Charts

• Statistics

• CSV reports

Phase 10 — Quality

• Validation

• Security review

• API testing

• UI testing

• Build verification

• Documentation

Do not jump directly to AI features.

---

19. TESTING REQUIREMENTS

At minimum verify:

• Student registration

• Login

• Invalid login

• JWT protection

• Admin authorization

• Eligibility calculation

• Duplicate application prevention

• Application status update

• Status history

• Interview creation

• Resource upload validation

• Notification creation

• Analytics calculations

• CSV export

• Frontend protected routes

• Responsive UI

• Backend and frontend production builds

---

20. SEED / DEMO DATA

Create realistic demo data:

• Admin account

• Multiple departments

• At least 10 students

• At least 5 companies

• Applications in different statuses

• Interview rounds

• Preparation PDFs represented by sample metadata

• Notifications

Never use real personal data.

---

21. AI FEATURES — FUTURE ONLY

Do NOT implement AI in the initial core build unless explicitly requested.

Future modules:

1. Interview Success Prediction

2. Student Risk Detection

3. Company Recommendation Engine

4. AI Resume Analyzer

These should later be isolated as an AI service/module so the core placement system remains stable.

---

22. DEFINITION OF DONE

A phase is complete only when:

• Code compiles/builds.

• Database migrations/schema work.

• APIs can be tested.

• Frontend pages load.

• Authentication/authorization works.

• No obvious console errors.

• Relevant validation exists.

• Documentation is updated.

• Existing features still work.

---

23. ALFRED BUILD PRINCIPLES

Build this like a real software product, not a collection of disconnected demo pages.

Priorities:

1. Correctness

2. Security

3. Maintainability

4. Clear UX

5. Demonstrability

6. Scalability

Keep the architecture simple enough for an MCA academic project while maintaining professional engineering practices.